Software Engineering Project Jan-2026

Milestone 2 Deliverables for Cargo-Core

Submitted By

Team Name: QuadCore-Devs

Team Code : Team-003

IITM Online BS Degree Program,
Indian Institute of Technology, Madras, Chennai
Tamil Nadu, India, 600036

Small Business Operations Platform for Rural Logistics Management

Project Details

Software Name: Cargo-Core

Department Context: Local-Logistic-Department

System Type: Small Business Logistics & Resource Management Platform

Date of Submission : 22-03-2026

Team Details

Team Name: QuadCore-Devs
Team Code: Team-003

Name	Roll Number	Role
Siddarth S	23f3001439	Product Manager / Scrum Master
YASHVARDHAN	23f2004644	Frontend / Code Reviewer
PRUTHVI PRASAD S	23f1002103	Frontend / Backend Dev
GAUTHAM KRISHNA S	23f2000466	Backend / Tester

DESIGN OF COMPONENTS

---

1. Customer Component *(Booking & Tracking APIs)*

- Submit a structured booking request with goods type, quantity, pickup and delivery location
- Receive a digital booking confirmation with order ID, cost breakdown and service details
- Get system-suggested vehicle size and labourer count based on cargo details
- Track real-time delivery status of active orders without calling the dispatcher
- Record full or partial payments and view remaining balance for each order
- View all past orders, quotes, and damage reports from the customer dashboard
- Submit damage reports with photo evidence and receive QR-coded acknowledgement
- Request price quotes before confirming a booking

---

2. Owner / Logistic Manager Component *(Finance, Monitoring & Operations APIs)*

- View auto-generated digital invoices for every completed order
- Track full, partial, and pending payments across all customers to monitor outstanding dues
- Monitor real-time vehicle locations across the fleet without calling drivers
- View daily, weekly, and monthly logistics reports and revenue dashboards
- Track labour attendance and working hours for accurate payroll calculation
- Generate weekly payroll summaries per labourer with transparent wage breakdowns
- Compare vehicle mileage against submitted fuel expenses to detect anomalies
- Prioritize and manage orders during peak season demand
- Monitor all communication between customers, drivers, and dispatcher in one place
- Manage warehouse registrations, fleet, zones, and user accounts

---

3. Dispatcher Component *(Resource Allocation & Scheduling APIs)*

- Receive AI-powered recommendations for vehicle type and labourer count per order
- Generate and share daily work schedules for labourers before each shift
- View estimated loading and unloading time predictions for each order
- Digitally confirm equipment issued and returned for each job to prevent disputes
- Manage pending dispatch queue and assign orders to available drivers
- Balance workload across drivers using load balancing and order clustering
- Track real-time driver status (active, idle, on-route, completed)

- Handle crisis situations such as vehicle breakdowns or driver unavailability
- Optimize delivery routes across multiple orders for minimum fuel usage
- Manage communication and coordination between warehouse and field teams

---

4. Warehouse Manager Component *(Inventory & Operations APIs)*

- Map and organize storage layout with sections, aisles, shelves, and bin locations
- Receive automatic low-stock alerts for packing materials and inventory items
- Record issued and returned equipment (crates, belts, blankets) per order
- Manage inbound orders and assign them to warehouse zones for picking
- Execute picking process generate pick lists, assign pickers, track progress per SKU
- Manage packing stations and track items packed per station per day
- Assign loading docks to trucks and track dock availability in real time
- Perform quality checks before dispatch (goods count, packaging, labelling, weight)
- Grade and process returned goods with condition assessment and refund calculation
- Monitor warehouse floor plan with zone-wise capacity and throughput metrics
- Track labourer availability and assign them to active orders

---

5. Driver Component *(Field Execution & Delivery APIs)*

- Receive complete job details including cargo type, pickup address, delivery address and OTP
- Get optimized navigation routes to minimize fuel usage and travel time
- Record digital start and end work times for transparent payment calculation
- Update delivery status at each stage (picked up, in-transit, delivered, failed)
- Sync delivery status updates even in low network or offline conditions
- Confirm equipment receipt and return at pickup and delivery sites
- View assigned order history and delivery performance stats

---

6. Inventory Management Component *(Backend APIs)*

- Create and manage inventory items per warehouse with SKU, category, unit, and location
- Track real-time stock levels and trigger safety stock alerts automatically
- Record every inventory movement (inbound, outbound, transfer) with order references
- Generate pick lists for orders based on available stock
- Create and manage restock requests with approval workflow
- Enforce warehouse-level inventory isolation for multi-warehouse operations

---

7. Labour & Workforce Component *(Backend APIs)*

- Register labourers and assign them to specific warehouses with skill tags
- Record daily attendance through digital check-in and check-out events
- Assign labourers to active orders based on skill and availability
- Track equipment issued to each labourer per job for accountability
- Generate weekly payroll data based on recorded attendance and work hours
- Monitor active vs inactive labourer availability in real time

---

8. Authentication & Access Control Component *(Auth APIs)*

- Register new users with role selection (Individual, Vendor, Warehouse Manager, Dispatcher)
- Login with email and password with JWT access and refresh token issuance
- Google OAuth 2.0 login for quick access
- Email OTP verification before completing signup
- Role-based dashboard redirection after successful login
- Forgot password and secure reset password via email token
- Token refresh and automatic session management
- Logout with token blacklisting via Redis

---

9. AI Intelligence Component *(AI & Analytics APIs)*

- Accept natural language questions from logistics managers about operations
- Generate and execute SQL queries from plain English using Gemini AI
- Return formatted, human-readable answers from live database data
- Maintain session-based conversation history for context-aware responses
- Classify query intent (database query, general question, greeting)
- Escalate unresolved or sensitive queries to a human agent automatically
- Track escalation status from open to resolved

---

10. Communication & Coordination Component *(Real-time APIs)*

- Provide a centralized chat system between dispatchers, drivers, and warehouse teams
- Send and receive messages within warehouse-specific communication threads
- Create and manage logistics alerts with severity levels and recommendations
- Track and resolve operational escalations raised by any team member
- Send notifications for order status changes, assignment updates, and alerts
- Log all communication for audit and dispute resolution purposes

System Design – Class Diagram

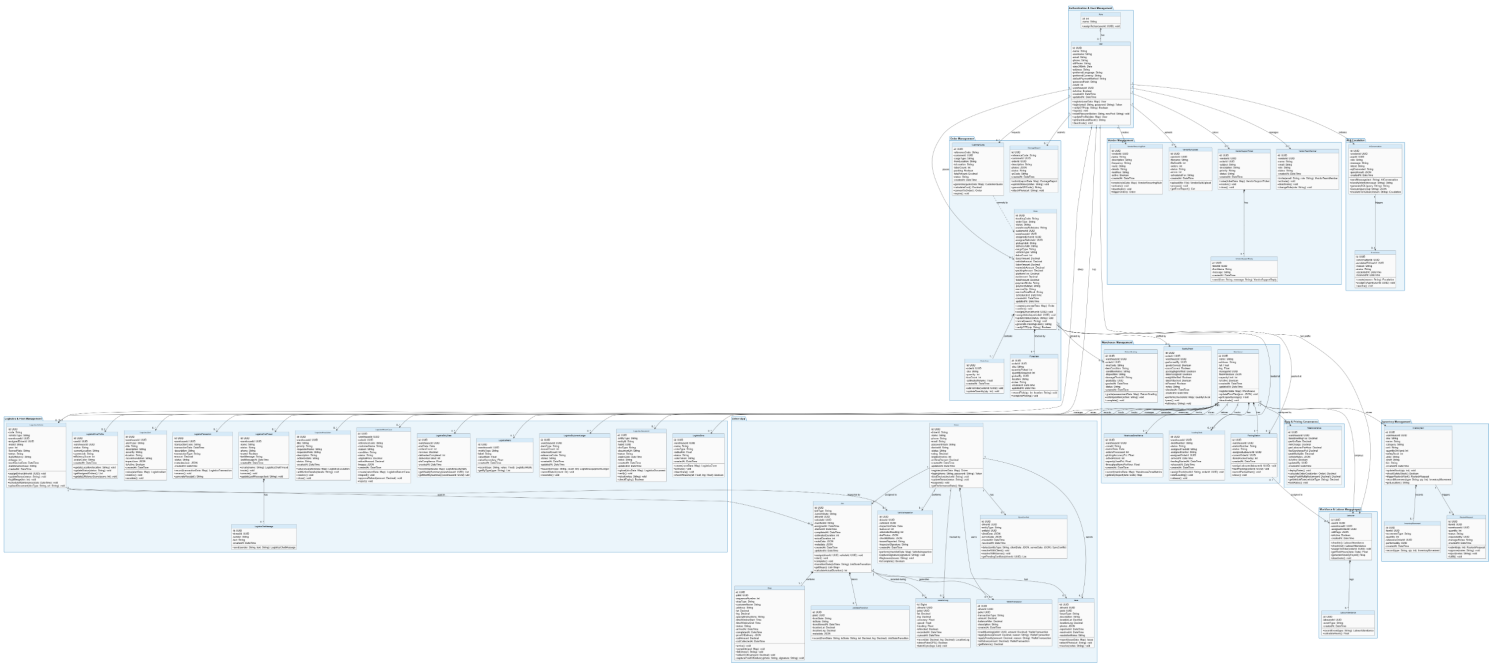
Overview

The class diagram represents the complete system architecture of CargoCore, consisting of **51 interconnected classes organized across 10 modules**. It illustrates the relationships between core components such as user management, order processing, warehouse operations, logistics, inventory, and vendor systems, ensuring a scalable and modular design.

Key Highlights

- Covers **10 major modules** including Authentication, Orders, Warehouse, Logistics, Vendor, and Driver App
- Clearly defines relationships between entities such as **User, Order, Warehouse, Driver, and Inventory**
- Supports **real-world logistics workflows** including tracking, dispatching, returns, and fleet management
- Designed for **scalability and modular development**

Click here to access the full detailed view of the CLASS DIAGRAM



Database Design – ER Diagram

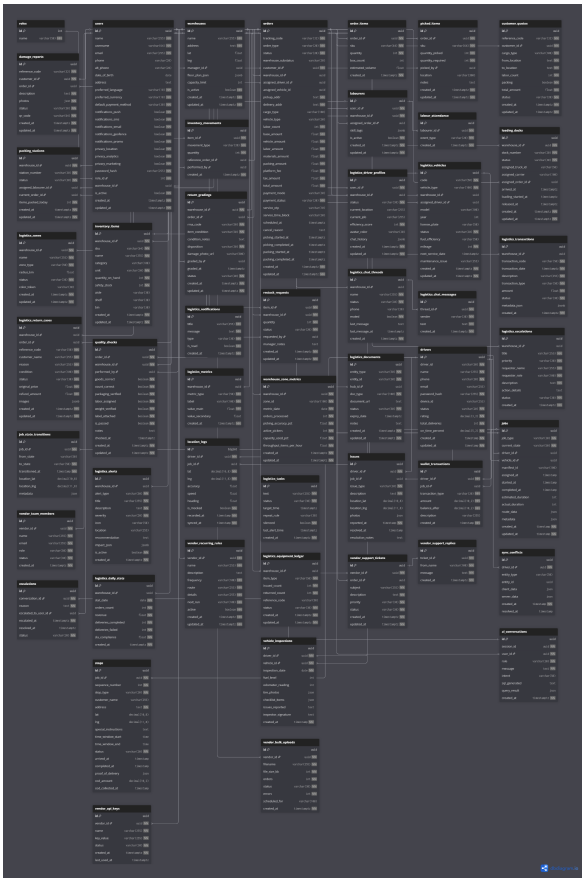
Overview

The ER (Entity-Relationship) diagram represents the database structure of the CargoCore system, defining entities, attributes, and relationships required for efficient data management. It captures how different components such as orders, users, warehouses, inventory, and logistics operations are interconnected at the database level.

Key Highlights

- Defines relationships such as:
    - **One-to-Many** (User → Orders, Warehouse → Inventory)
    - **Many-to-One** (Orders → Drivers, Inventory → Warehouse)
- Ensures **data integrity and consistency** across all modules
- Supports **complex logistics operations** including order tracking, inventory flow, and delivery management
- Optimized for **efficient querying and scalability**

Click here to access the full detailed view of the ER DIAGRAM



PROJECT SCHEDULE

Date	Phase / Focus	Main Agenda Points (key items only)	Expected Outcome / What's Mostly Done
Before Feb 22	Milestone 1- Identify User Requirement	a. Identify users (primary, secondary, tertiary) b. Conduct interviews/observations to identify pain points (no direct feature suggestion) c. Record audio/video proof of research d. Write SMART user stories: As a [user], I want [action], so that [benefit] e. Compile deliverables (users, research, pain points, user stories in PDF)	Milestone 1 completed and ready for submission
Feb 21	Milestone 1 closure + Milestone 2 kickoff	a. Final verification of all Milestone 1 deliverables b. Final "sanity check" of documentation and transcripts c. Kickoff discussion for Milestone 2 d. Record meeting minutes (attendees, decisions, actions)	Milestone 2 started, minutes captured
Feb 23	Pre-Milestone 2 alignment & UI/tech prep	a. Finalize project branding (name, tagline, themes, colors) b. Confirm tech stack & open-source libraries c. Review digitized high-fidelity wireframes in Figma d. Validate UI designs against Milestone 1 user stories e. Record meeting minutes	Branding & tech stack locked, wireframes validated & approved, minutes captured
March 1	Full project scheduling & alignment	a. Finalize complete project timeline till April 19 deadline b. Create/export Gantt chart + high-level timeline (compressed) c. Define 2 short sprints clearly d. Set up Trello/Jira board + initial tasks e. Finalize tools/tech stack (Jira/Trello tracking, GitHub issues/PRs, Draw.io diagrams, Flask backend, Vue/React frontend, pytest, etc.) f. Record meeting minutes	Full A-to-Z schedule ready (ends April 19), Gantt created, board live, tools finalized, minutes captured

March 8	Milestone 2 design + UI	a. Review full schedule/Gantt (take/export screenshots for PDF) b. Finish class diagram + system components description c. Finish DB design (ER diagram/schema) d. Frontend: demo most linked UI pages (navigable) e. Export Kanban board screenshot for PDF f. Record meeting minutes	Design & UI ~80–90% done, Gantt + Kanban screenshots ready, minutes captured
March 14	Milestone 2 polish + Sprint 1 prep	a. Final Milestone 2 check (diagrams, UI screenshots, zip/readme) b. Prepare PDF sections (schedule, Gantt/Kanban screenshots, diagrams, UI pages, minutes so far) c. Sprint 1 priority APIs/user stories & assignments d. Risk discussion (tight timeline → plan daily async standups via Slack if needed) e. Record meeting minutes	Milestone 2 95% ready, PDF sections drafted, minutes captured
March 21	Milestone 2 closure + Milestone 3 kickoff	a. Final verification of all Milestone 2 deliverables b. Final "sanity check" of code/documentation c. Kickoff Milestone 3 (Sprint 1: API dev) d. Record meeting minutes	Milestone 2 submitted, Sprint 1 started, minutes captured
March 28	Sprint 1 – API progress & mid-check	a. Standup: progress last week b. Demo 4–6 API endpoints (Postman/YAML) c. Review PRs/issues d. Update board & Gantt e. Log blockers/bugs as GitHub issues (with screenshots for final report) f. Record meeting minutes	~50% Sprint 1 APIs done, issues tracked, minutes captured
April 4	Sprint 1 – Wrap-up & testing focus	a. Final Sprint 1 demos (all APIs) b. YAML completeness & user story mapping check c. Run pytest suite (show results/screenshots) d. Log all bugs as GitHub issues (with status updates) e. Quick retro + Sprint 2 handoff f. Record meeting minutes	Sprint 1 complete: APIs + YAML + basic tests ready, issues tracked, minutes captured
April 11	Sprint 2 kickoff – Integration & testing start	a. Kickoff Sprint 2: connect frontend-backend b. Assign integration + test tasks c. Plan test cases matrix (input/expected/actual) d. Update board with Sprint 2 backlog e. Quick refresh (CORS, auth if needed) f. Record meeting minutes	Sprint 2 started, first integrations attempted, minutes captured

April 18	Sprint 2 – Final testing, polish & submission prep	a. Demo full integrated flows b. Show test cases (4–5 APIs) + pytest results c. Highlight failures + fixes (document for report) d. Gather quick user feedback (document summary + plan for final) e. Final README check (include hosting info if deployed, e.g., Render) f. Compile final PDF report (M2 + Sprint 1 + Sprint 2 + tools + GitHub issue screenshots) g. Record draft video demo (5–10 min overview + user stories) h. Start slides (overview, user stories, design/teamwork) i. Dry-run demo & submit prep j. Record meeting minutes	App stable, tests ready, report/video/slides draft done, minutes captured → Submit by April 19
April 19	Milestone 5 (Final submission)	a.Verify and do a final sanity check of the whole project b.Record presentation video ,Finalize video (edit/upload) c.Finish PDF report (proofread, add run instructions) - Final code zip + README (install/run steps for Ubuntu/Mac + Windows) - d.Submit all (zip, PDF, video, slides)	The whole Software engineering project including all the code zip, documentation, milestones submitted in the submission drive

Milestone 2 Plan (Scheduling and Design)

Duration: February 21 – March 21, 2026 (≈1 month)

Focus: Create full project schedule, produce design artifacts (components, class/DB diagrams), complete most UI pages (linked/navigable), prepare PDF report + frontend zip.

Week	Dates (approx)	Main Goals / Key Tasks	Saturday Meeting Outcome / What's Mostly Done
1	Feb 21 – Feb 28	- Final Milestone 1 verification & sanity check - Kickoff Milestone 2 discussion - Finalize branding (LogiFlow, tagline, colors) - Confirm tech stack (React + Tailwind, etc.) - Review/validate Figma wireframes against user stories - Setup GitHub repo & initial board	Feb 21: Milestone 2 started Feb 23: Branding/tech/wireframes finalized & validated
2	Mar 1 – Mar 7	- Finalize overall project timeline till April 19 - Create/export Gantt chart & high-level timeline - Define Sprint 1 & Sprint 2 structure - Set up Trello/Jira board + populate initial tasks - Finalize tools stack (Jira, GitHub, Draw.io, etc.) - Start high-level system components description	March 1: Full A-to-Z schedule ready, Gantt created, board live, tools finalized
3	Mar 8 – Mar 14	- Review & screenshot Gantt + Kanban board - Finish class diagram + relationships - Finish DB design (ER diagram / schema) - Develop & demo most UI pages (connected, redirection implemented) - Prepare frontend zip + README (run instructions) - Draft PDF sections (schedule, diagrams, UI screenshots, minutes)	March 8: Design & UI ~80–90% done, screenshots ready March 14: Milestone 2 pieces 95% ready, PDF sections drafted
4	Mar 15 – Mar 21	- Final check: diagrams updated, UI linked, zip/readme complete - Sanity check code/documentation - Compile full Milestone 2 PDF report - Record final minutes & prepare submission	March 21: All Milestone 2 deliverables verified & ready for submission

Sprint 1 (Milestone 3: API Development)

Duration: March 22 – April 4 (≈2 weeks)

Focus: Build core APIs fast — prioritize 6–8 essential ones.

Week	Dates	Main Goals / Key Tasks	Saturday Meeting Outcome
1	Mar 22 – Mar 28	- Select priority APIs - Design/implement 3–4 endpoints (YAML first) - Basic error handling	March 28: Demo early APIs (~50%), issues logged
2	Mar 29 – Apr 4	- Finish remaining APIs - Complete YAML + user story mapping - Write/run basic pytest	April 4: Full demo, pytest screenshots, bugs tracked

Sprint 2 (Milestone 4: Testing & Integration)

Duration: April 5 – April 18 (≈2 weeks)

Focus: Quick integration + essential testing — focus on 4–6 key flows.

Week	Dates	Main Goals / Key Tasks	Saturday Meeting Outcome
1	Apr 5 – Apr 11	- Connect frontend to backend (core flows first) - Start detailed test cases - Fix initial bugs	April 11: First integrated demos
2	Apr 12 – Apr 18	- Complete integrations - Document test cases + pytest (show failures/fixes) - Quick user feedback (document summary) - Update README	April 18: Full app demo, tests ready, feedback noted

Milestone 5 Plan (Final Submission)

Duration: April 18 (Saturday evening) – April 19 (Sunday)

Focus: Quick compile, verify, record & submit by April 19 deadline.

Day	Dates	Main Goals / Key Tasks	Outcome by End
1	April 18 (Saturday meeting)	- Demo full working app (prototype) - Quick review of tests + pytest screenshots - Compile final PDF report (copy M2 + Sprint 1 APIs/YAML + Sprint 2 tests/feedback + tools + GitHub issue screenshots) - Record draft video demo (5–10 min overview + user stories) - Start slides (project overview, user stories, design/teamwork)	Report 70–80% done, draft video + slides ready, app verified
2	April 19 (Sunday – async)	- Finish PDF report (proofread, add run instructions) - Final code zip + README (install/run steps for Ubuntu/Mac + Windows) - Finalize video (edit/upload) - Complete slides - Dry-run presentation/demo - Submit all (zip, PDF, video, slides)	Everything submitted by April 19

Sprint Schedule

❖ Sprint 1: Milestone 1 – Identify User Requirements

➢ Identify users of the application (primary, secondary, tertiary)
➢ Conduct interviews/observations to identify pain points (no direct feature suggestion)
➢ Record proof (audio/video snippets)
➢ Write SMART user stories:
As a [user], I want [action], so that [benefit]
➢ Prepare deliverables (PDF: users, research, pain points, user stories)
➢ **Date: (Before 22/02/2026)**

❖ Sprint 2: Milestone 2 – Planning & Design

➢ Finalize project branding, tech stack, and tools
➢ Create full project schedule + Gantt chart
➢ Setup Jira/Trello board and tasks
➢ Design system architecture (components, class diagram)
➢ Create database schema (ER diagram)
➢ Develop and link major UI screens
➢ Prepare Milestone 2 PDF (diagrams, UI, schedule, screenshots)
➢ **Date: 21/02/2026 - 21/03/2026**

❖ Sprint 3: Milestone 3 – API Development (Sprint 1)

➢ Identify and prioritize core APIs (6–8 endpoints)
➢ Develop APIs with proper request/response structure (YAML)
➢ Implement validation and error handling
➢ Map APIs to user stories
➢ Perform basic testing using pytest
➢ Track bugs/issues using GitHub
➢ **Date: 22/03/2026 - 04/04/2026**

❖ Sprint 4: Milestone 4 – Integration & Testing (Sprint 2)

➢ Integrate frontend with backend APIs
➢ Implement core user flows
➢ Design and execute test cases (input/expected/actual)
➢ Fix integration and functional bugs
➢ Document test results and feedback
➢ Update README and project documentation
➢ **Date: 05/04/2026 - 18/04/2026**

❖ Sprint 5: Milestone 5 – Final Testing & Report Preparation

➢ Demo complete working application
➢ Run final pytest and capture results
➢ Document failures and fixes
➢ Collect and summarize user feedback
➢ Compile final PDF report (all milestones + screenshots)
➢ Prepare slides and draft video demo
➢ **Date: 18/04/2026**

❖ Sprint 6: Final Submission

➢ Final sanity check of project (code + documentation)
➢ Finalize video recording and upload
➢ Complete report proofreading and run instructions
➢ Prepare final code zip + README
➢ Submit all deliverables (PDF, code, video, slides)
➢ **Date: 19/04/2026**

SCRUM Meetings Schedule and Minutes

SCRUM Meetings: Conducted on scheduled dates via Google Meet (evening sessions)

Minutes from February 8, 2026 SCRUM Meeting:

- Conducted team introductions and discussed individual roles and expertise
- Analyzed multiple project domains and finalized *Local Logistics / Delivery System*
- Brainstormed logistics workflows using the hub-and-spoke model
- Identified key pain points such as package loss, lack of tracking, and manual inefficiencies

---

Minutes from February 11, 2026 SCRUM Meeting:

- Reviewed team progress and clarified individual responsibilities
- Discussed compliance requirements including audio/video consent
- Introduced SMART guidelines for writing user stories
- Explored additional features such as transcription and GenAI integration

---

Minutes from February 17, 2026 SCRUM Meeting:

- Identified and finalized primary, secondary, and tertiary users
- Analyzed interview data and documented key pain points
- Drafted 21 SMART user stories covering the complete workflow
- Discussed storyboard design and finalized documentation approach
- Initiated AI usage report and centralized log collection

Minutes from February 21, 2026 SCRUM Meeting:

- Conducted final verification of all Milestone 1 deliverables
- Completed sanity check of documentation, transcripts, and user stories
- Finalized AI usage report and consolidated deliverables
- Initiated Milestone 2 with focus on frontend wireframing and UI structure

---

Minutes from February 23, 2026 SCRUM Meeting:

- Finalized project branding (*LogiFlow*), theme, and color palette
- Confirmed technology stack (React, Tailwind CSS, etc.)
- Reviewed and validated high-fidelity UI wireframes
- Approved UI designs for development phase

Minutes from March 1, 2026 SCRUM Meeting:

- Reviewed frontend components and verified feature completeness
- Identified missing UI elements and proposed enhancements
- Defined testing strategy using dummy data
- Confirmed database schema progress and frontend readiness

Minutes from March 5, 2026 SCRUM Meeting:

- Presented frontend demo and received client feedback
- Established formal code review process
- Updated Kanban board and aligned tasks with timeline
- Received approval on tech stack and backend priorities

Minutes from March 8, 2026 SCRUM Meeting:

- Reviewed full project schedule and exported Gantt chart
- Completed class diagram and system architecture
- Finalized ER diagram and database schema
- Demonstrated UI pages with navigation and responsiveness
- Captured Kanban board and project screenshots

Minutes from March 14, 2026 SCRUM Meeting:

- Verified Milestone 2 deliverables including UI and diagrams
- Prepared report sections with screenshots and documentation
- Planned Sprint 1 APIs and assigned tasks
- Discussed risks and contingency strategies

Minutes from March 21, 2026 SCRUM Meeting:

- Conducted final sanity check of Milestone 2 deliverables
- Verified frontend implementation and documentation
- Initiated API development phase (Sprint 1)
- Defined core APIs, testing strategy, and responsibilities

Project Scheduling Tools

To manage the project effectively, we used visual task management and scheduling tools to organize work across different phases and ensure timely completion of milestones.

Tools Used:

- **Kanban Board** (for task management and tracking)
- **GitHub Issues** (for bug tracking and issue logging)
- **Gantt Chart (Excel)** (for project timeline visualization)

Description:

We primarily used a **Kanban board** to manage and track the progress of tasks throughout the project. Tasks were categorized into columns such as *To Do*, *In Progress*, and *Completed*, allowing the team to clearly visualize the workflow and monitor progress at each stage of development. This improved transparency and ensured that all team members were aware of their responsibilities.

For issue tracking during development and testing, **GitHub Issues** was used to log bugs, blockers, and improvements. Each issue was documented properly and tracked until resolution, which also helped in maintaining a record for reporting and evaluation purposes.

Additionally, a **Gantt chart** was created to represent the complete project schedule from February to April. This provided a high-level view of milestones, deadlines, and dependencies, ensuring that all phases such as design, API development, integration, and testing were completed on time.

Outcome:

These tools helped in:

- Efficient tracking of tasks and progress
- Clear visualization of workflow
- Proper documentation of issues
- Timely completion of all milestones

Project Gantt Chart

Overview

The Gantt chart represents the complete timeline of the project from **Milestone 1 (User Requirement Analysis)** to **Final Submission (April 19, 2026)**. It provides a visual representation of all major phases, including planning, design, development, integration, testing, and final delivery.

The chart illustrates how the project is divided into multiple milestones and sprints, ensuring a structured and systematic approach to development.

Description

The project begins with **Milestone 1**, which focuses on identifying user requirements through interviews, observations, and formulation of SMART user stories. This phase lays the foundation for all subsequent development activities.

Following this, **Milestone 2** covers planning and design activities, including project scheduling, Gantt chart creation, system design (class diagrams and database schema), and UI development. This phase ensures that the architecture and workflow of the system are clearly defined before implementation begins.

The development phase is carried out in **Sprint 1 (Milestone 3)**, where core APIs are designed, implemented, and tested. Tasks are divided into initial API development and completion with testing, ensuring incremental progress.

Next, **Sprint 2 (Milestone 4)** focuses on integrating the frontend and backend components. This phase includes implementing core user flows, conducting detailed testing, fixing bugs, and collecting feedback.

Finally, **Milestone 5** involves final testing, report preparation, video demonstration, and submission of all deliverables.

Key Features of the Gantt Chart

- Clearly shows the **timeline of all milestones and sprints**
- Represents **parallel and overlapping tasks** (e.g., UI development and system design)
- Highlights **dependencies between phases** (design → development → testing → submission)
- Ensures **efficient time management and task tracking**
- Helps in monitoring progress and meeting deadlines

Outcome

The Gantt chart played a crucial role in:

- Planning the project effectively
- Managing task dependencies and overlaps
- Tracking progress across milestones and sprints
- Ensuring timely completion of the project within the deadline

---

Gantt Chart Task Table

Task	Start Date	End Date
Milestone 1 - User Requirement Analysis	**15-02-2026**	**22-02-2026**
Identify Users & Conduct Interviews	**15-02-2026**	**20-02-2026**
User Stories & Documentation	**20-02-2026**	**22-02-2026**
Milestone 2 - Planning & Design	**21-02-2026**	**21-03-2026**
Branding, Tech Stack & Wireframes	**21-02-2026**	**23-02-2026**
Project Planning & Gantt Creation	**01-03-2026**	**07-03-2026**

System Design (Class Diagram + DB Design)	**08-03-2026**	**14-03-2026**
UI Development & Linking	**08-03-2026**	**14-03-2026**
Documentation & Finalization (Milestone 2)	**15-03-2026**	**21-03-2026**
Sprint 1 - API Development (Milestone 3)	**22-03-2026**	**04-04-2026**
API Design & Initial Development	**22-03-2026**	**28-03-2026**
API Completion & Testing	**29-03-2026**	**04-04-2026**
Sprint 2 - Integration & Testing (Milestone 4)	**05-04-2026**	**18-04-2026**
Frontend-Backend Integration	**05-04-2026**	**11-04-2026**
Testing, Bug Fixing & Feedback	**12-04-2026**	**18-04-2026**
Milestone 5 - Final Submission	**18-04-2026**	**19-04-2026**
Final Report, Video & Slides Preparation	**18-04-2026**	**19-04-2026**
Final Submission	**19-04-2026**	**19-04-2026**

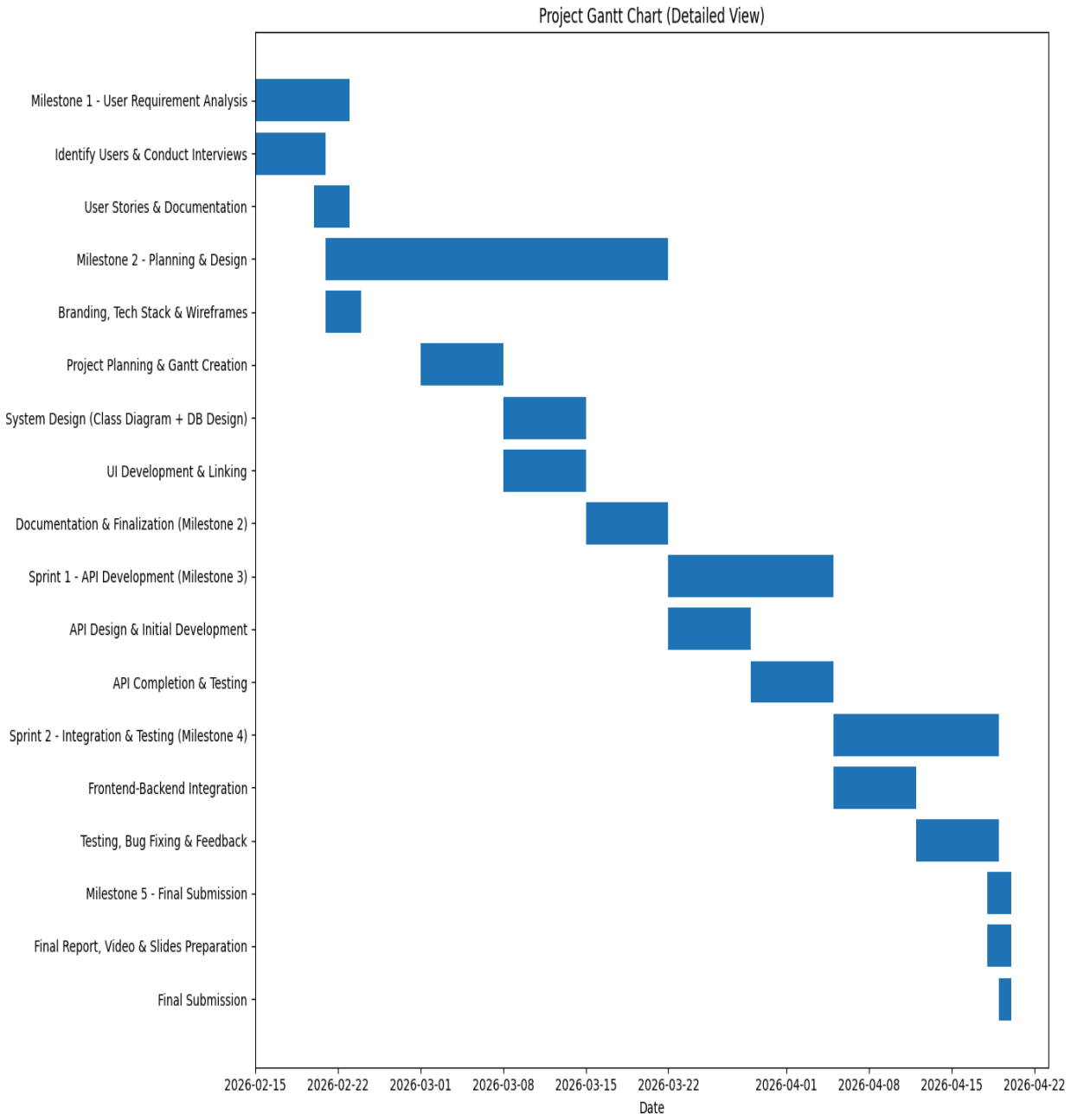


Fig: Project Gantt Chart showing the timeline of milestones, sprints, and tasks from February to April 2026.

Project Tracking Mechanism

(Kanban Board)

Overview

To effectively manage and track project progress, a **Kanban board** was used as the primary project tracking tool. It provided a visual representation of tasks across different stages of development, ensuring transparency and efficient workflow management.

Description

The Kanban board was structured into multiple columns representing task status:

- **Backlog** – Tasks that are planned but not yet started
- **To Do** – Tasks ready to be picked up
- **In Progress** – Tasks currently being worked on
- **In Review** – Tasks under review or testing
- **Done** – Completed tasks

Each task was represented as a card containing details such as task title, priority, and assignee. Tasks were continuously moved across columns as progress was made.

The board also included:

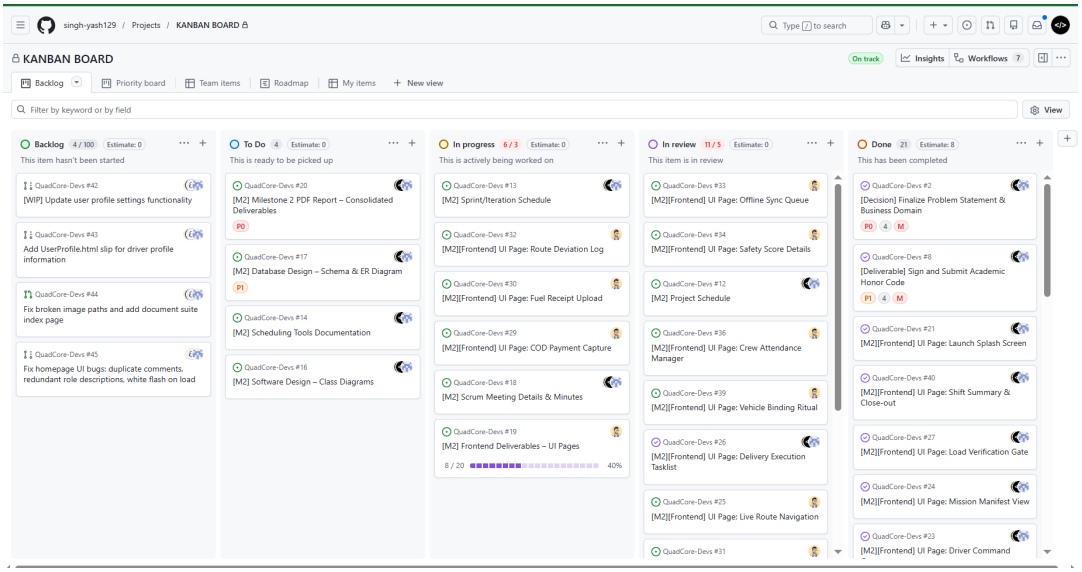
- Task prioritization (P0, P1, etc.)
- Assignment of responsibilities to team members
- Real-time progress tracking
- Integration with GitHub for issue tracking

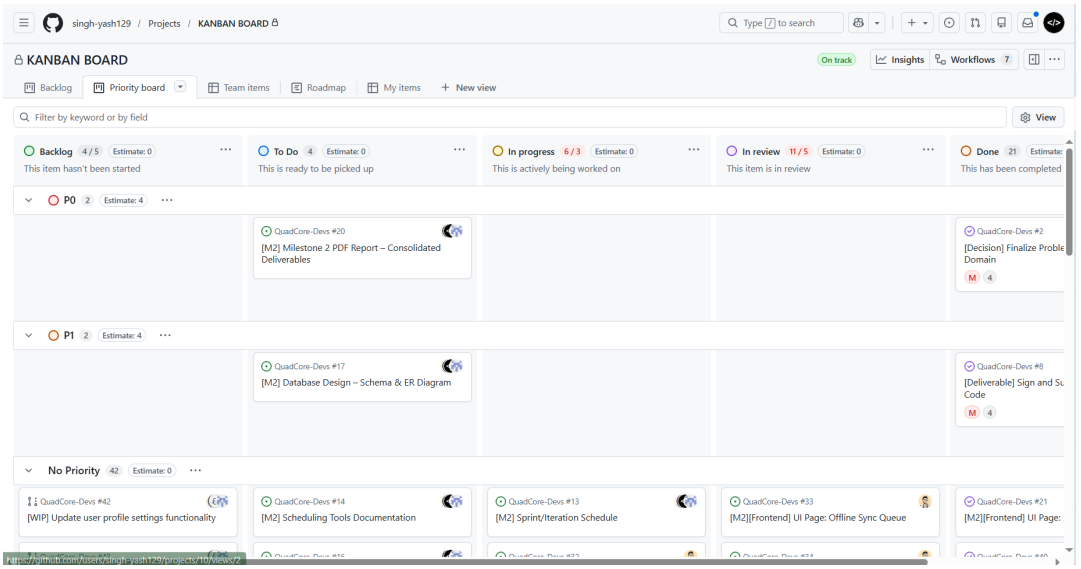
Outcome

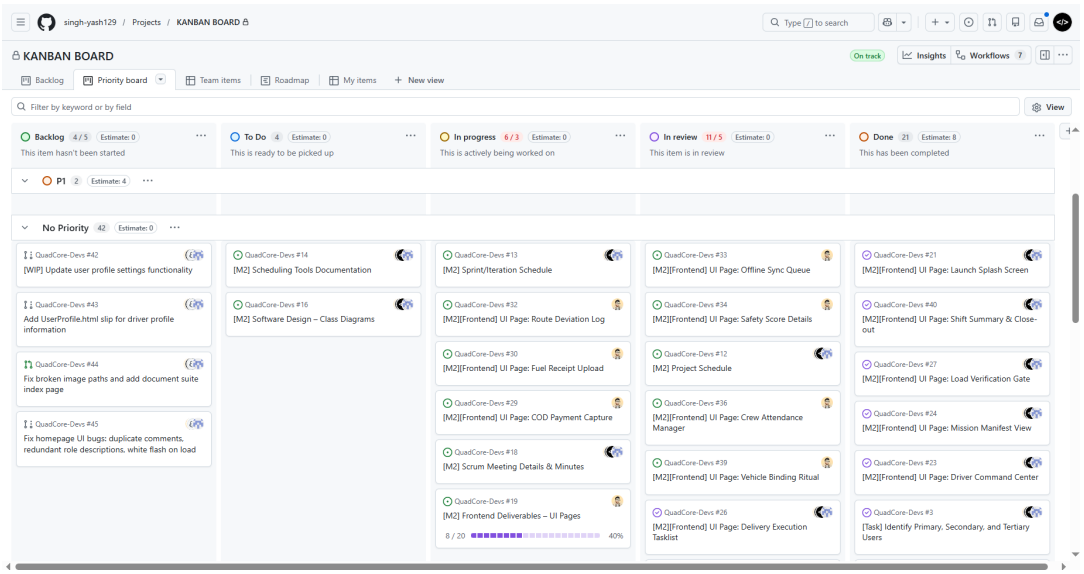
The Kanban board helped in:

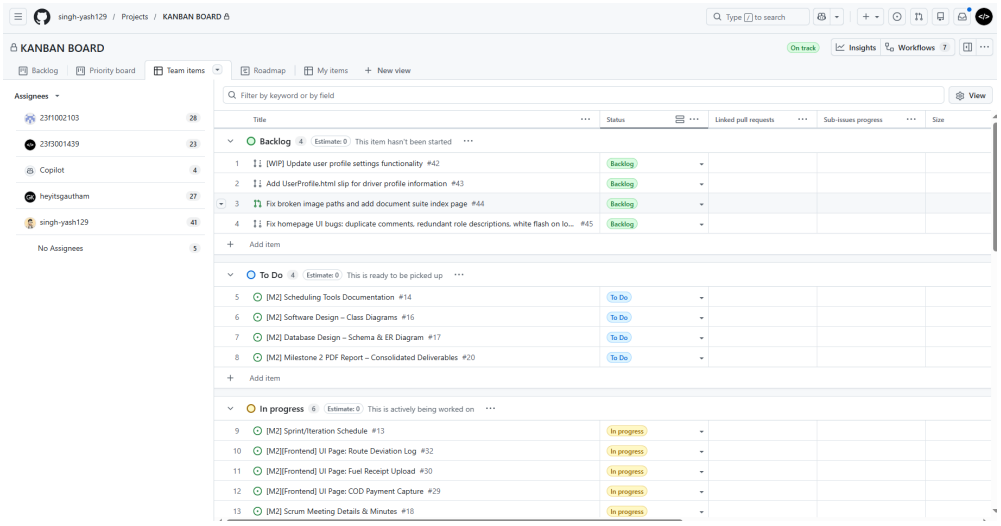
- Efficient task tracking and management
- Clear visualization of project progress
- Better team coordination and accountability
- Timely completion of milestones and deliverables

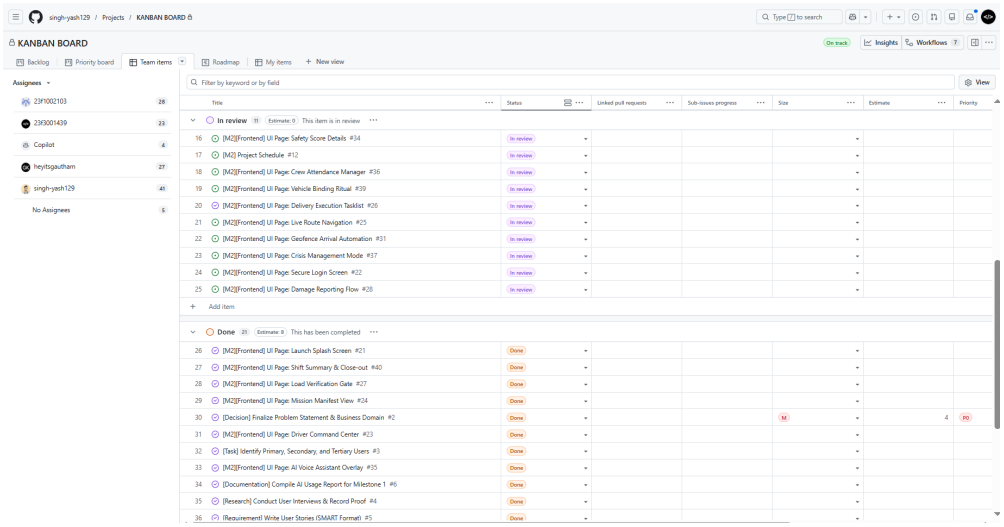
Figure : Kanban Board showing task distribution and progress tracking across different stages

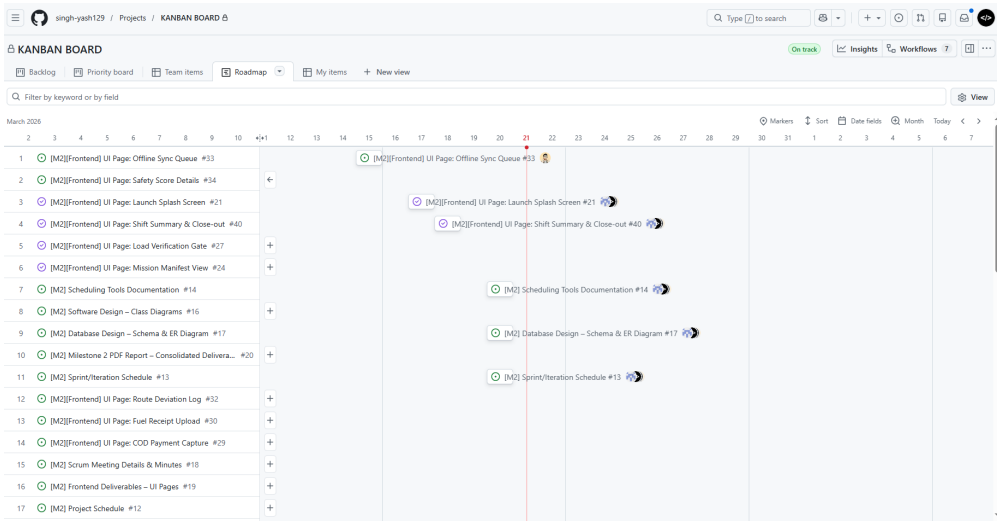


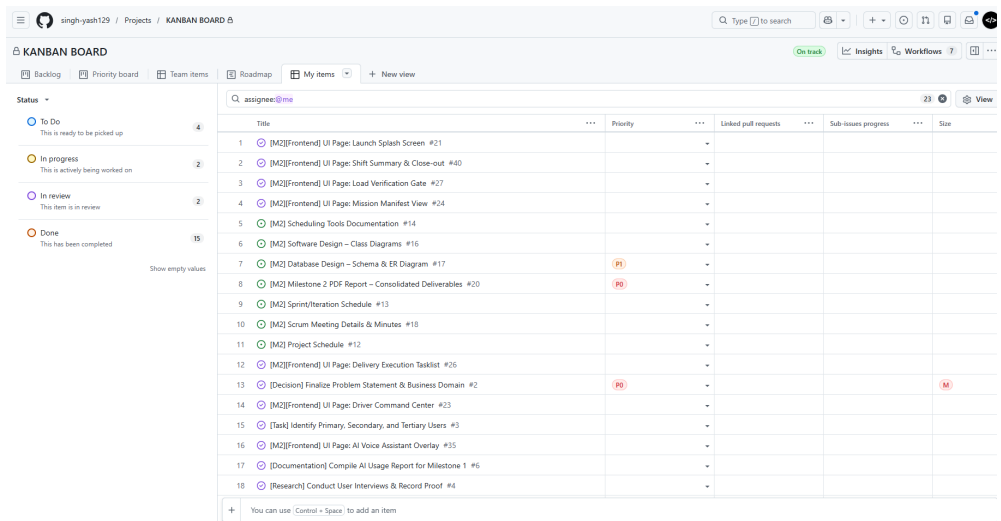





singh-yash129 / Projects / KANBAN BOARD
KANBAN BOARD
On track
Insights
Workflows 7
Backlog
Priority board
Team items
Roadmap
My items
New view
Assignees
23f1002103 28
23f3001439 23
Copilot 4
heyitsgautham 27
singh-yash129 41
No Assignees 5
Filter by keyword or by field
View
Title
Status
Linked pull requests
Sub-issues progress
Size
Backlog 4 Estimate: 0 This item hasn't been started
1 [WIP] Update user profile settings functionality #42 Backlog
2 Add UserProfile.html slip for driver profile information #43 Backlog
3 Fix broken image paths and add document suite index page #44 Backlog
4 Fix homepage UI bugs: duplicate comments, redundant role descriptions, white flash on lo... #45 Backlog
Add item
To Do 4 Estimate: 0 This is ready to be picked up
5 [M2] Scheduling Tools Documentation #14 To Do
6 [M2] Software Design – Class Diagrams #16 To Do
7 [M2] Database Design – Schema & ER Diagram #17 To Do
8 [M2] Milestone 2 PDF Report – Consolidated Deliverables #20 To Do
Add item
In progress 6 Estimate: 0 This is actively being worked on
9 [M2] Sprint/Iteration Schedule #13 In progress
10 [M2][Frontend] UI Page: Route Deviation Log #32 In progress
11 [M2][Frontend] UI Page: Fuel Receipt Upload #30 In progress
12 [M2][Frontend] UI Page: COD Payment Capture #29 In progress
13 [M2] Scrum Meeting Details & Minutes #18 In progress


singh-yash129 / Projects / KANBAN BOARD
KANBAN BOARD
Backlog
Priority board
Team items
Roadmap
My items
New view
Assignees
23f1002103 28
23f3001439 23
Copilot 4
heyitsgautham 27
singh-yash129 41
No Assignees 5
Filter by keyword or by field
View
Title
Status
Linked pull requests
Sub-issues progress
Size
Estimate
Priority
In review 11 Estimate: 0 This item is in review
16 [M2][Frontend] UI Page: Safety Score Details #34 In review
17 [M2] Project Schedule #12 In review
18 [M2][Frontend] UI Page: Crew Attendance Manager #36 In review
19 [M2][Frontend] UI Page: Vehicle Binding Ritual #39 In review
20 [M2][Frontend] UI Page: Delivery Execution Tasklist #26 In review
21 [M2][Frontend] UI Page: Live Route Navigation #25 In review
22 [M2][Frontend] UI Page: Geofence Arrival Automation #31 In review
23 [M2][Frontend] UI Page: Crisis Management Mode #37 In review
24 [M2][Frontend] UI Page: Secure Login Screen #22 In review
25 [M2][Frontend] UI Page: Damage Reporting Flow #28 In review
Add item
Done 21 Estimate: 0 This has been completed
26 [M2][Frontend] UI Page: Launch Splash Screen #21 Done
27 [M2][Frontend] UI Page: Shift Summary & Close-out #40 Done
28 [M2][Frontend] UI Page: Load Verification Gate #27 Done
29 [M2][Frontend] UI Page: Mission Manifest View #24 Done
30 [Decision] Finalize Problem Statement & Business Domain #2 Done M 4 P0
31 [M2][Frontend] UI Page: Driver Command Center #23 Done
32 [Task] Identify Primary, Secondary, and Tertiary Users #3 Done
33 [M2][Frontend] UI Page: AI Voice Assistant Overlay #35 Done
34 [Documentation] Compile AI Usage Report for Milestone 1 #6 Done
35 [Research] Conduct User Interviews & Record Proof #4 Done
36 [Requirement] Write User Stories (SMART Format) #5 Done


singh-yash129 / Projects / KANBAN BOARD
KANBAN BOARD
On track
Insights
Workflows 7
Backlog
Priority board
Team items
Roadmap
My items
New view
Filter by keyword or by field
View
March 2026
Markers
Sort
Date fields
Month
Today
1 [M2][Frontend] UI Page: Offline Sync Queue #33
2 [M2][Frontend] UI Page: Safety Score Details #34
3 [M2][Frontend] UI Page: Launch Splash Screen #21
4 [M2][Frontend] UI Page: Shift Summary & Close-out #40
5 [M2][Frontend] UI Page: Load Verification Gate #27
6 [M2][Frontend] UI Page: Mission Manifest View #24
7 [M2] Scheduling Tools Documentation #14
8 [M2] Software Design – Class Diagrams #16
9 [M2] Database Design – Schema & ER Diagram #17
10 [M2] Milestone 2 PDF Report – Consolidated Delivera... #20
11 [M2] Sprint/Iteration Schedule #13
12 [M2][Frontend] UI Page: Route Deviation Log #32
13 [M2][Frontend] UI Page: Fuel Receipt Upload #30
14 [M2][Frontend] UI Page: COD Payment Capture #29
15 [M2] Scrum Meeting Details & Minutes #18
16 [M2] Frontend Deliverables – UI Pages #19
17 [M2] Project Schedule #12
[M2][Frontend] UI Page: Offline Sync Queue #33
[M2][Frontend] UI Page: Launch Splash Screen #21
[M2][Frontend] UI Page: Shift Summary & Close-out #40
[M2] Scheduling Tools Documentation #14
[M2] Database Design – Schema & ER Diagram #17
[M2] Sprint/Iteration Schedule #13


singh-yash129 / Projects / KANBAN BOARD
KANBAN BOARD
On track
Insights
Workflows 7
Backlog
Priority board
Team items
Roadmap
My items
New view
Status
To Do
This is ready to be picked up 4
In progress
This is actively being worked on 2
In review
This item is in review 2
Done
This has been completed 15
Show empty values
assignee:@me
23
View
Title
Priority
Linked pull requests
Sub-issues progress
Size
1 [M2][Frontend] UI Page: Launch Splash Screen #21
2 [M2][Frontend] UI Page: Shift Summary & Close-out #40
3 [M2][Frontend] UI Page: Load Verification Gate #27
4 [M2][Frontend] UI Page: Mission Manifest View #24
5 [M2] Scheduling Tools Documentation #14
6 [M2] Software Design – Class Diagrams #16
7 [M2] Database Design – Schema & ER Diagram #17 P1
8 [M2] Milestone 2 PDF Report – Consolidated Deliverables #20 P0
9 [M2] Sprint/Iteration Schedule #13
10 [M2] Scrum Meeting Details & Minutes #18
11 [M2] Project Schedule #12
12 [M2][Frontend] UI Page: Delivery Execution Tasklist #26
13 [Decision] Finalize Problem Statement & Business Domain #2 P0 M
14 [M2][Frontend] UI Page: Driver Command Center #23
15 [Task] Identify Primary, Secondary, and Tertiary Users #3
16 [M2][Frontend] UI Page: AI Voice Assistant Overlay #35
17 [Documentation] Compile AI Usage Report for Milestone 1 #6
18 [Research] Conduct User Interviews & Record Proof #4
You can use Control + Space to add an item

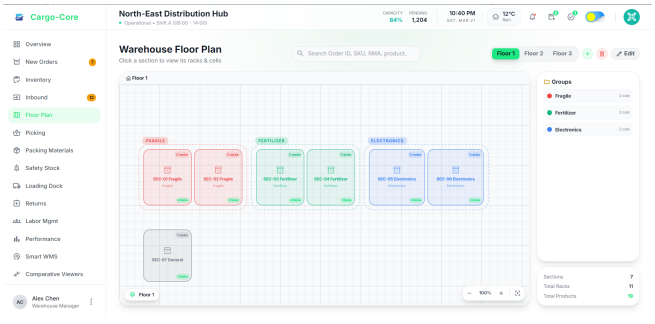
Frontend Interface Screens

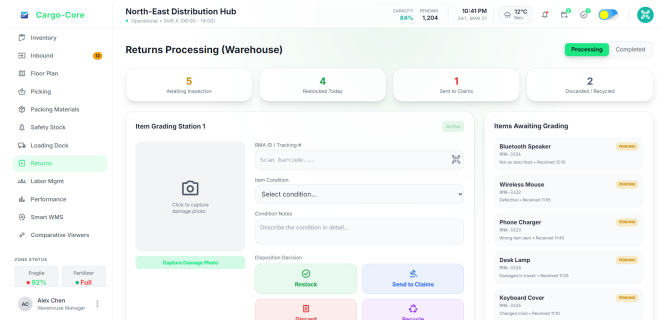
Overview

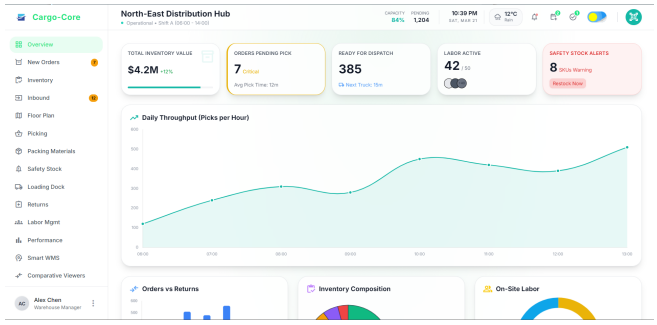
The frontend of the application is designed to support multiple user roles, each with a dedicated dashboard tailored to their specific responsibilities. The interfaces are designed to be intuitive, responsive, and aligned with the workflows identified during user research.

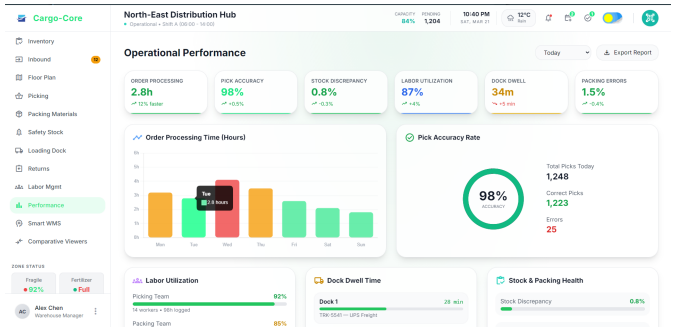
Warehouse Manager Dashboard

The Warehouse Manager dashboard provides an overview of inventory, active orders, and warehouse operations. It enables monitoring of stock levels, tracking incoming and outgoing shipments, and managing warehouse activities efficiently.



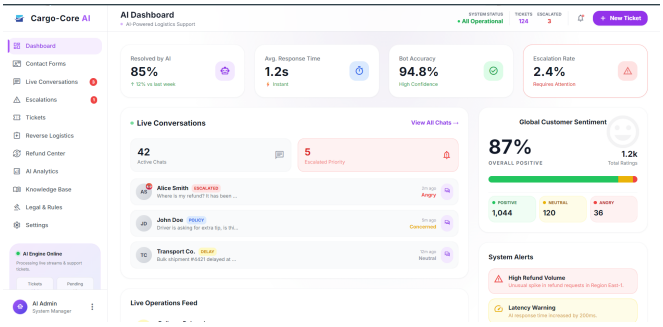


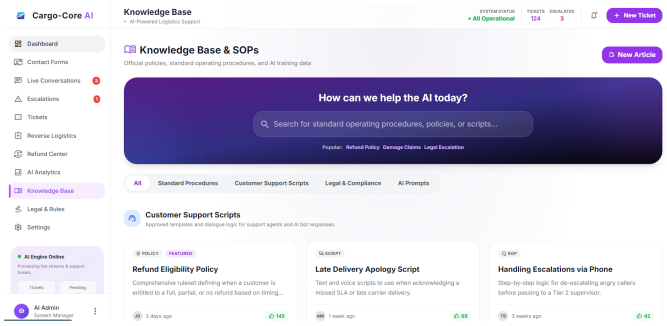


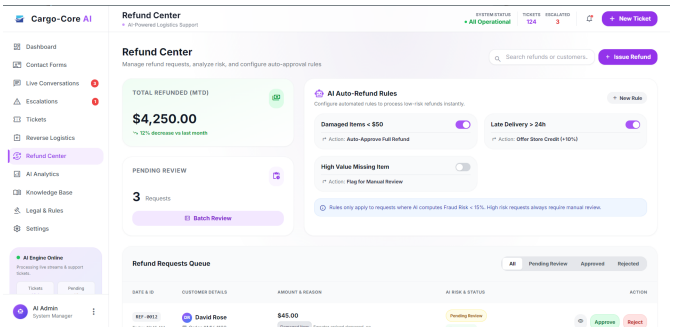


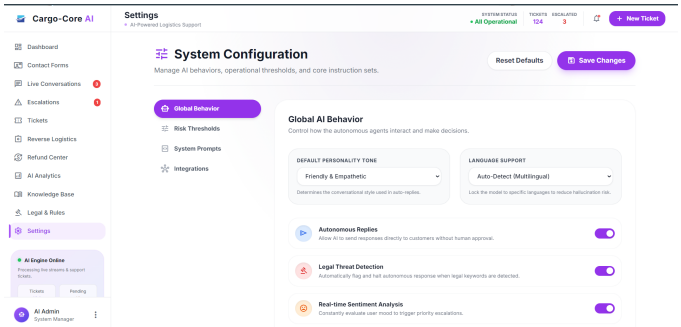
Customer Support Dashboard

The Customer Support dashboard allows support agents to handle user queries, complaints, and issue resolution. It provides access to order details, customer interactions, and support tickets for efficient problem handling.



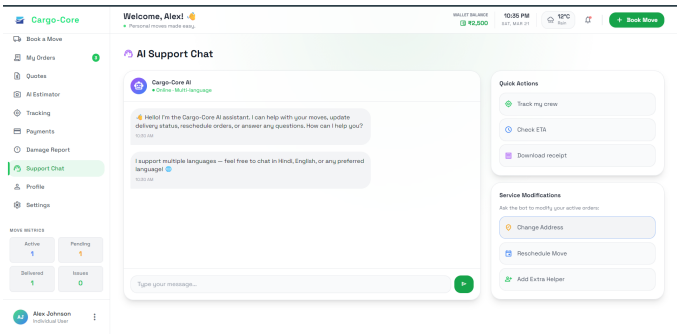


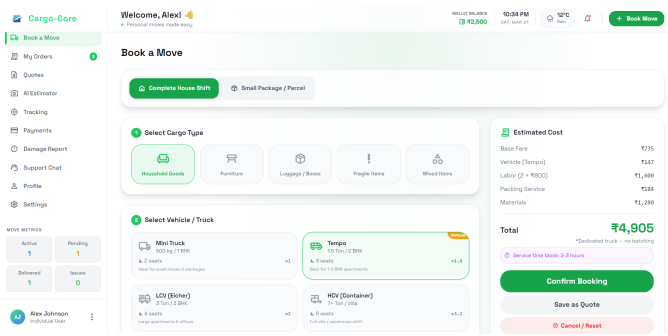


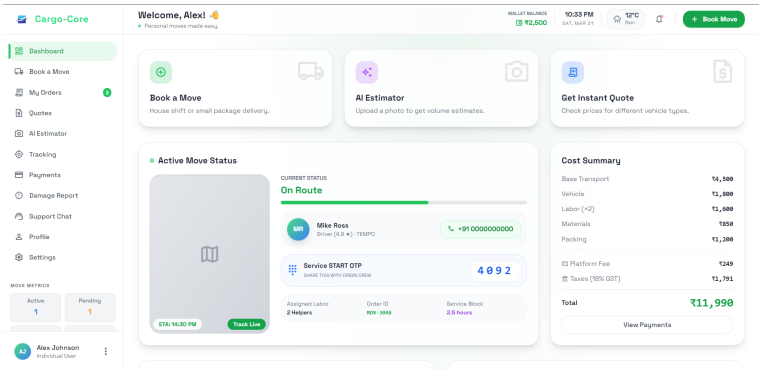


Customer Dashboard

The Customer dashboard enables users to place orders, track deliveries, and view order history. It provides real-time updates on shipment status and ensures a smooth user experience for managing deliveries.

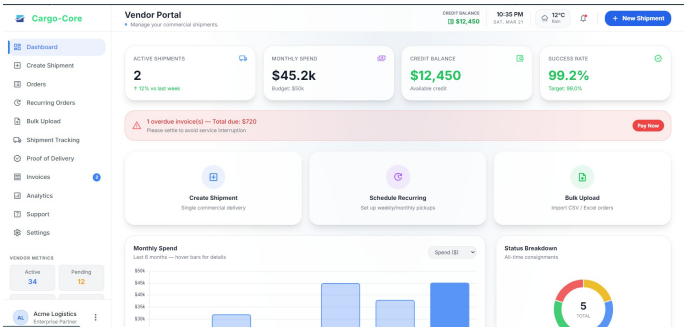


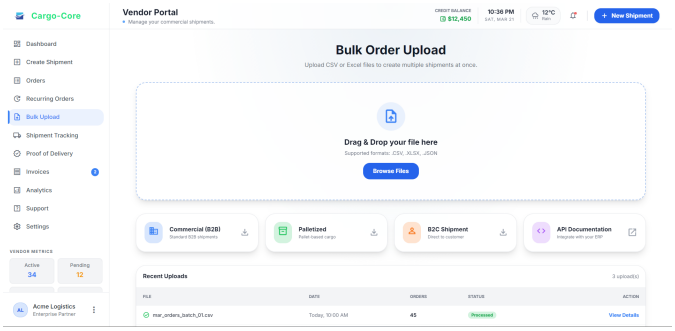


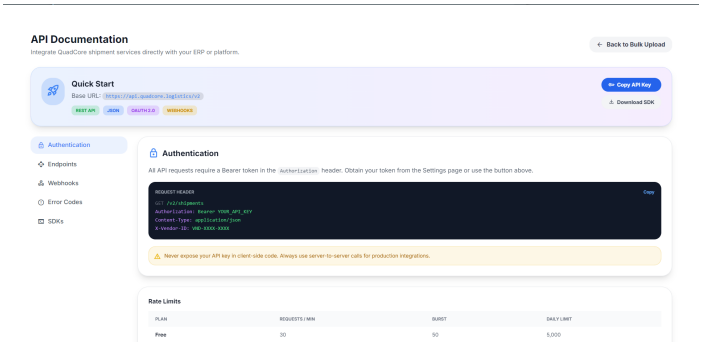


Vendor Dashboard

The Vendor dashboard allows vendors to manage inventory, update product or shipment details, and monitor order requests. It facilitates coordination between vendors and the logistics system.

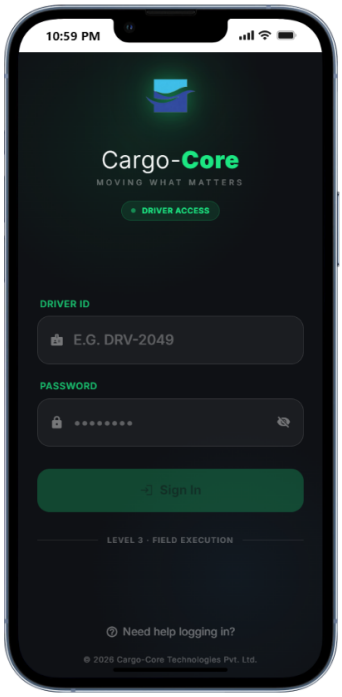


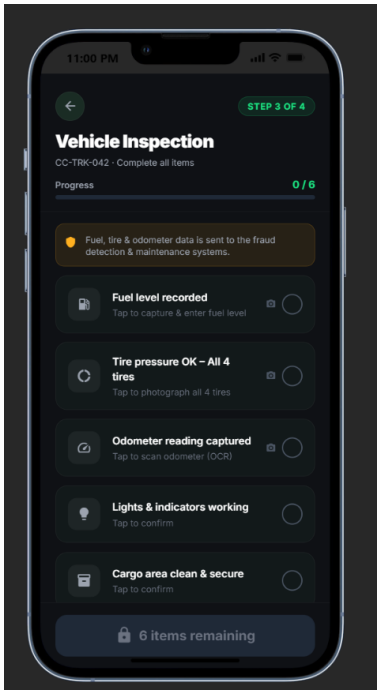


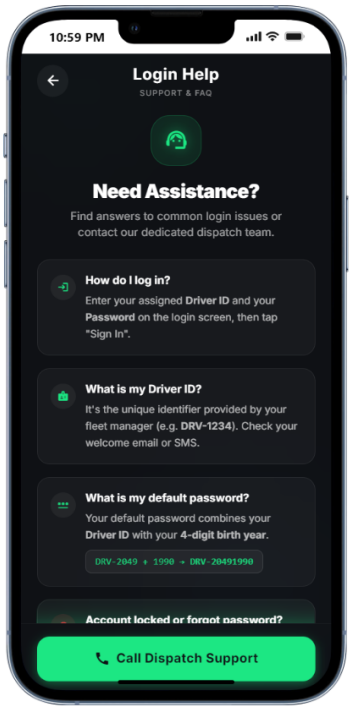


Driver Dashboard

The Driver dashboard provides delivery personnel with route details, assigned deliveries, and navigation support. It helps drivers update delivery status and manage tasks efficiently in real time.

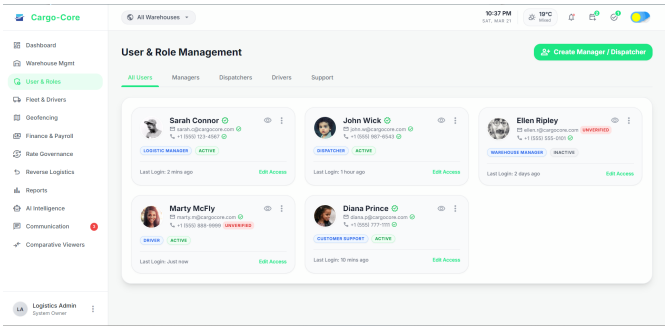


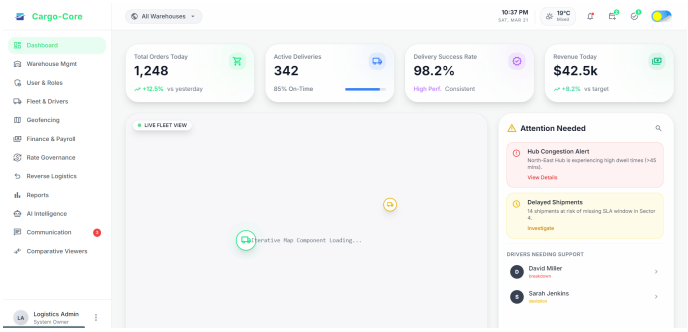


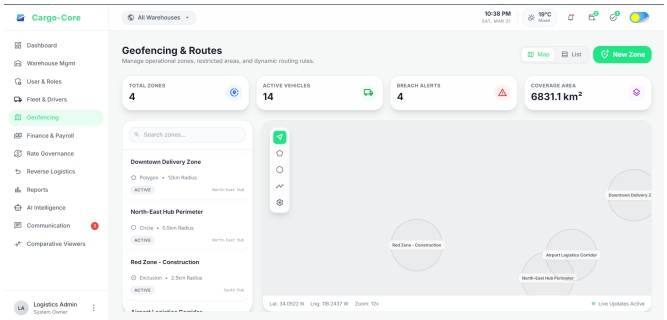


Logistics Manager Dashboard

The Logistics Manager dashboard provides a high-level view of overall logistics operations.
It includes monitoring of deliveries, fleet management, and performance tracking across the system.







Dispatcher Dashboard

The Dispatcher dashboard enables assignment of deliveries to drivers and route planning. It ensures efficient coordination between warehouse, drivers, and delivery schedules.